

DSA STUDY BLUEPRINT SERIES

82. Top View of a Binary Tree

Printing Vertical Columns (Top View)

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Vertical Line:** Group nodes by horizontal offset coordinates from root (0).
- Use queue-based BFS tracking offsets to resolve top-most values.

Memory & Execution Blueprint

Horizontal Offsets: Left child (-1) • Root (0) • Right child (1)

C++ Implementation Reference

Standard code layout and syntax

```
// BFS queue stores pair (node, offset).  
// Map offset coordinates to first seen value: map topViewMap.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N \log N)$ (Map keys)
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** Using `unordered_map` + `min/max` tracks coordinates in $O(N)$ time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.